2

Software Architecture

Architectural Styles

Includes content adapted from *Fundamentals of Software Architecture – An Engineering Approach*, 2020, O'Reilley

2.7

Event-Driven Architecture Style

Popular asynchronous architecture style with high performance and scalability

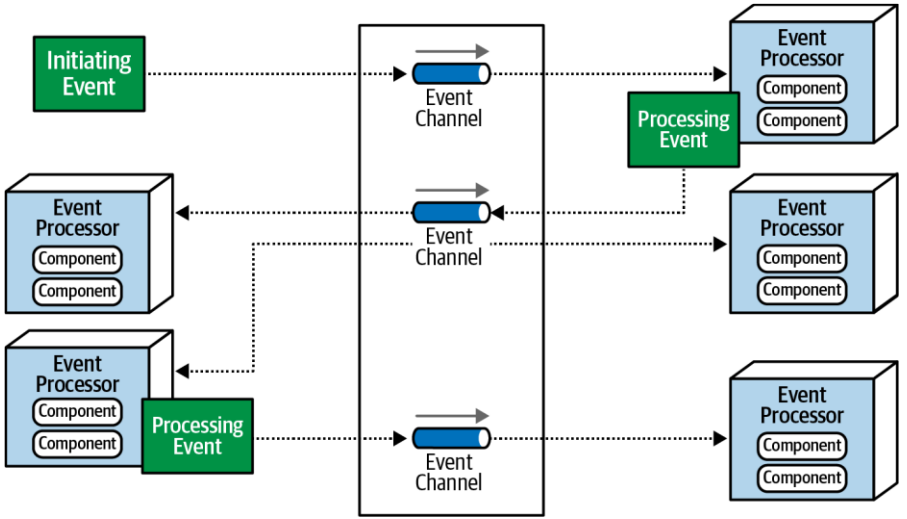


Event-Driven Architecture Style

- Made up of decoupled event processing components that asynchronously receive and process events.
  - Capable of high-performance
  - Potential for large scalability
  - High adaptability and modularity
  - Can appear on its own or in combination with another architectural style (e.g. event-driven microservices).
- There are two primary topologies:
  - **Broker topology** – Event processing is not controlled centrally
  - **Mediator topology** – Event processing is encapsulated and centralized

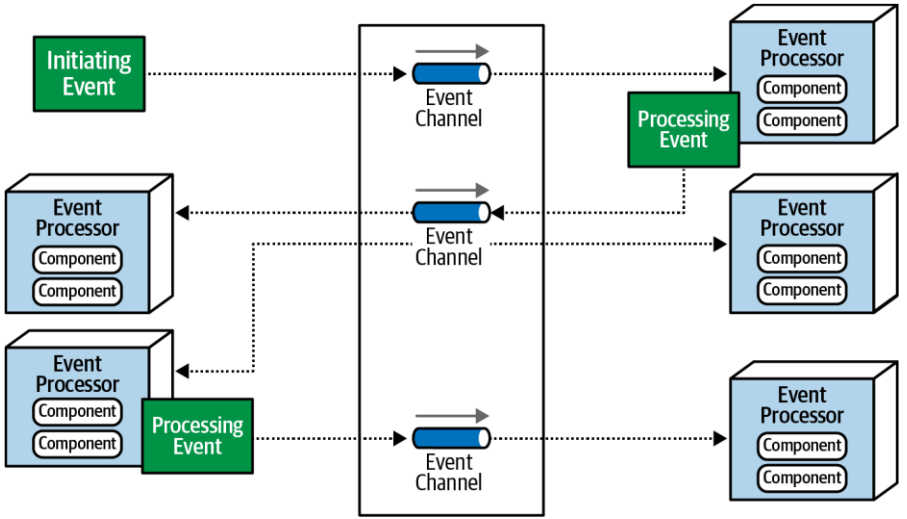
Broker Topology

- There is no central mediator coordinating (orchestrating) the response to events.
- Flow is chain-broadcasted to event processors using a lightweight message broker (RabbitMQ, ActiveMQ, HornetMQ,...).
- Suitable for simple event processing flows.
- No centralized event coordination or orchestration.



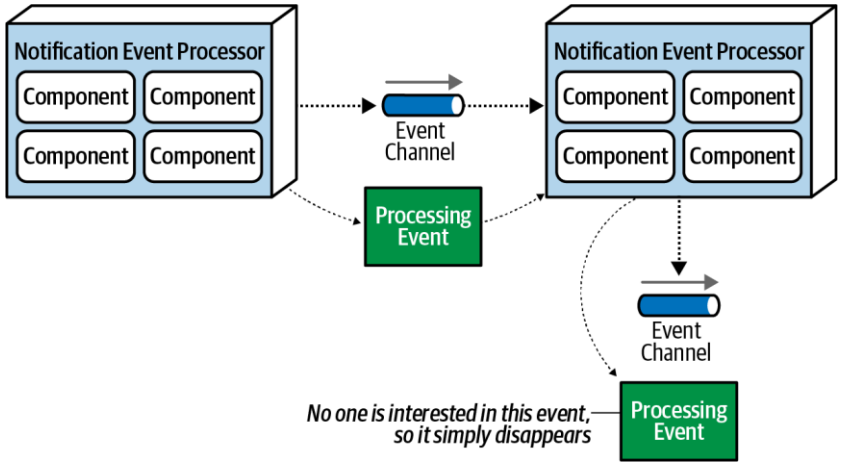
Broker Topology

- There are four primary architectural components at play:
  - **Initiating event** – The initial event that starts event processing flow.
  - **Event broker**–The event broker has multiple event channels, where specific events are advertised asynchronously, usually in a publish/subscribe model. It is usually federated (runs multiple domain-related instances)
  - **Event processor** – Event processors statically monitor specific event channels, accepting and processing new events.
  - **Processing Event** – An event that signals that a prior event has been successfully processed. By chaining multiple Processing Events and Event processors an event processing flow is obtained.



Broker Topology

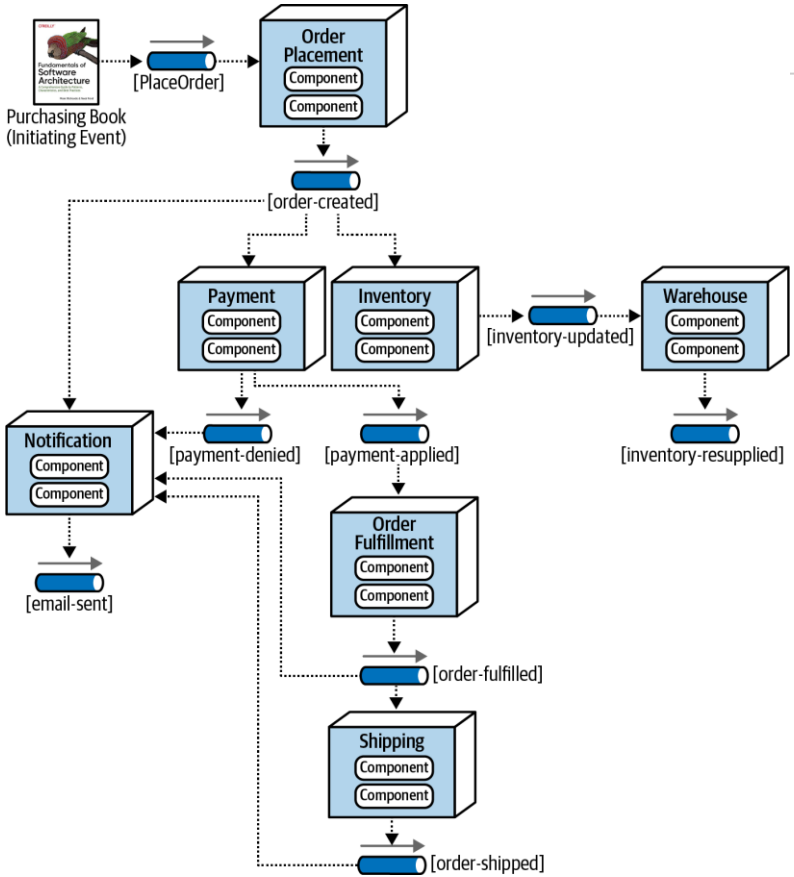
- Common practice is for all event processors to advertise their processing events, even if no one cares about them.
  - This sets up the system for architectural extensibility.
- Event Processors are highly decoupled and communicate asynchronously, so they can be scaled individually.



Broker Topology - Example

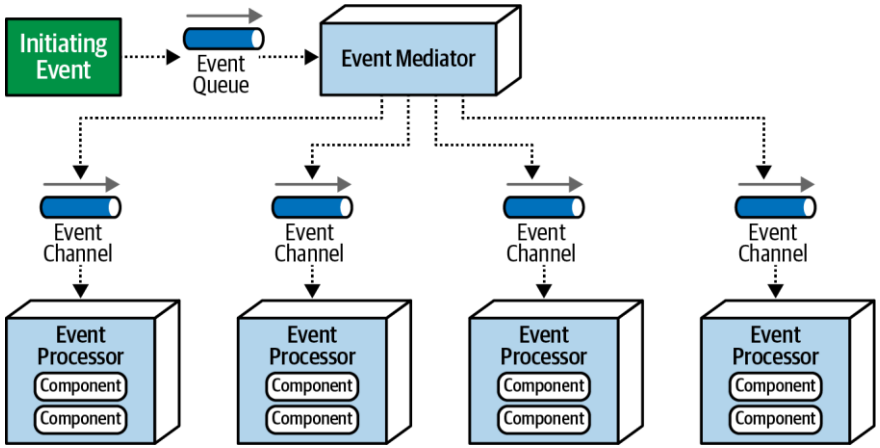
- Retail order system:
- Initiating event: PlaceOrder

Event Processor	Input Event(s)	Processing Event(s)
OrderPlacement	PlaceOrder	Order-created
Payment	Order-created	Payment-denied, Payment-applied
Inventory	Order-created	Inventory-updated
Warehouse	Inventory-updated	Inventory-resupplied
Notification	Order-created, Payment-denied, Order-fulfilled, Order-shipped	Email-sent
Order Fulfilment	Payment-applied	Order-fulfilled
Shipping	Order-fulfilled	Order-shipped



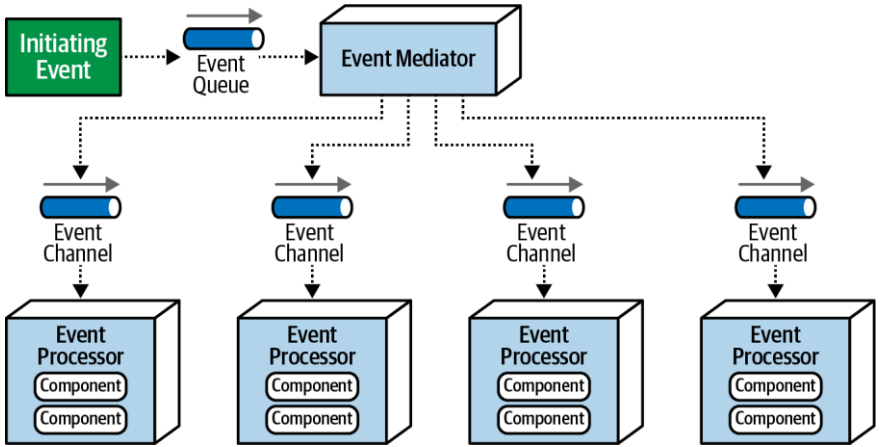
Mediator Topology

- An event mediator manages and controls the processing flow of initiating events by coordinating the event processors.
- Architectural Components are:
  - **Initiating Event** – An event that starts the whole process.
  - **Event Queue** – A queue of initializing events, consumed by the event mediator.
  - **Event Mediator** – Knows the desired workflow for each initializing event and generates the corresponding Processing Events. Usually replicated within domain.
  - **Event Channel** – Usually point-to-point queues that allow the event mediator to send a Processing Event to the corresponding Event Processor.
  - **Event Processor** – Will respond to Processing Events by performing the necessary processing. *(do not generate events themselves as in broker topology)*



Mediator Topology

- Mediator is usually replicated to eliminate single point of failure and improved throughput and performance.
- Mediators are usually associated with a specific domain or group of events (federated):
  - Improves decoupling and scaling potential.
  - For example, Customer Service Mediator, Order Management Mediator.
- Events can be interpreted differently in Broker and Mediator topologies:
  - **Broker Topology** – An event is an information describing that something has happened (e.g. *order-created*).
  - **Mediator Topology** – An event is an instruction or command to do something (e.g. *create-order*).
  - Events can be ignored in broker topologies, but are never ignored in mediator topologies.



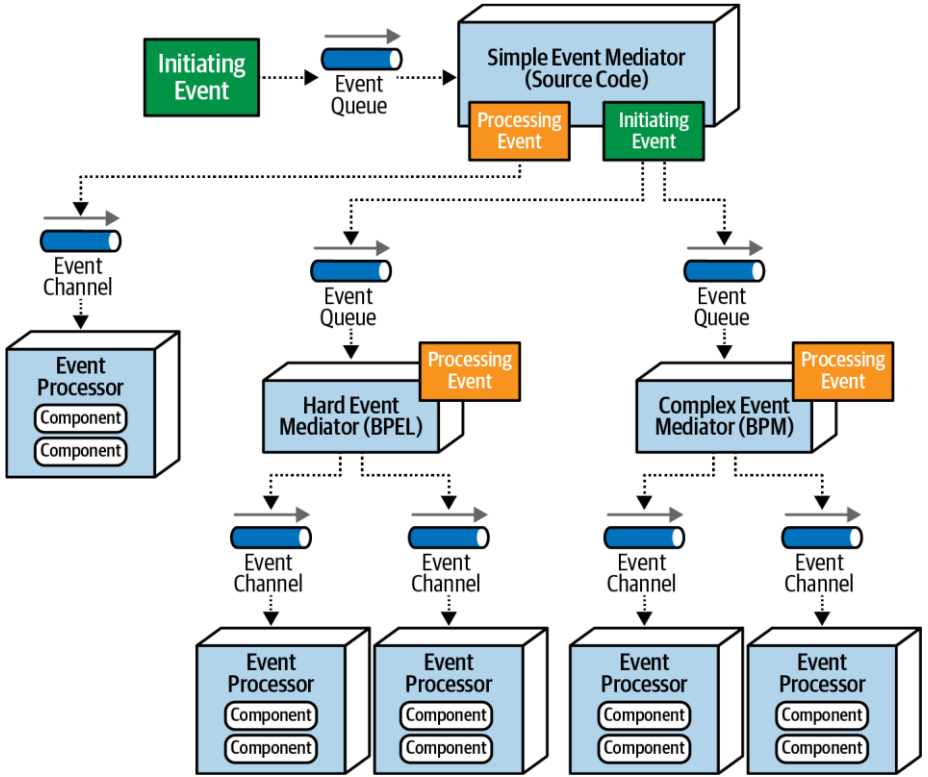


Mediator Topology – Frameworks and Libraries

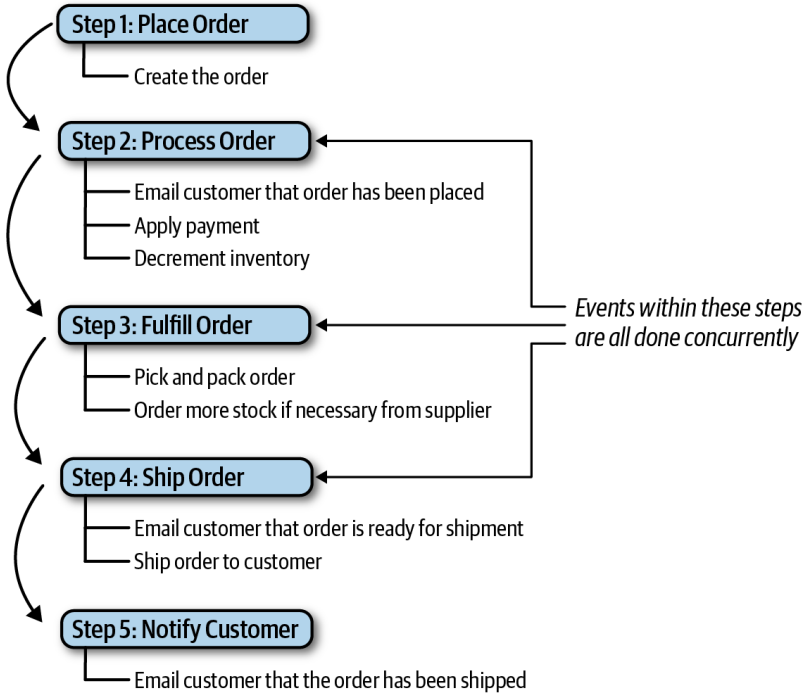
- Many frameworks exist for supporting the development of mediators, depending on the complexity and type of processes being supported:
  - **For scenarios with simpler error handling and orchestration needs:** Apache Camel, Mule ESB, Spring Integration would be good solutions.
  - **For scenarios with more complex workflows:** Apache ODE, Oracle BPEL Process Manager offer more capabilities.
    - These are based on BPEL (Business Process Execution Language), an XML format describing the steps, involved in event processing, as well as including information for error-handling, redirection, multicasting and other capabilities.
  - **For scenarios with human interactions and long-running transactions:** A Business Process Manager (BPM) engine, such as jBPM would be preferrable.
- Choosing the right tool for the job is important:
  - Using Camel for processing workflows with complex events and human interactions would be very difficult and hard to maintain.
  - Conversely, using jBPM for simple flows would entail a significant waste of effort.

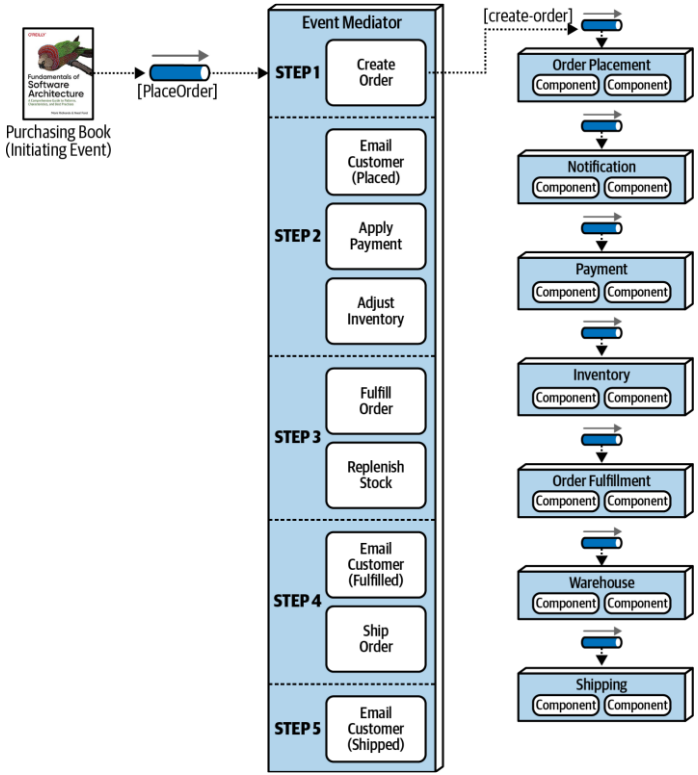
Mediator Topology – Event Delegation

- Events with different classes of complexity can often be found within the same system.
- In this case, it is possible to include distinct types of mediators in the system:
  - A simple mediator analyses the initiating event and delegates processing to a mediator of suitable complexity.

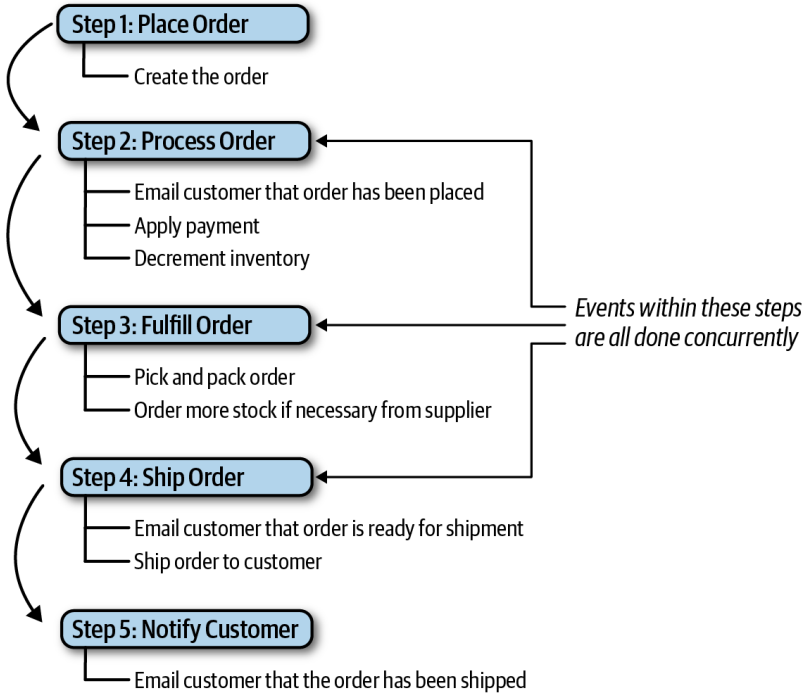


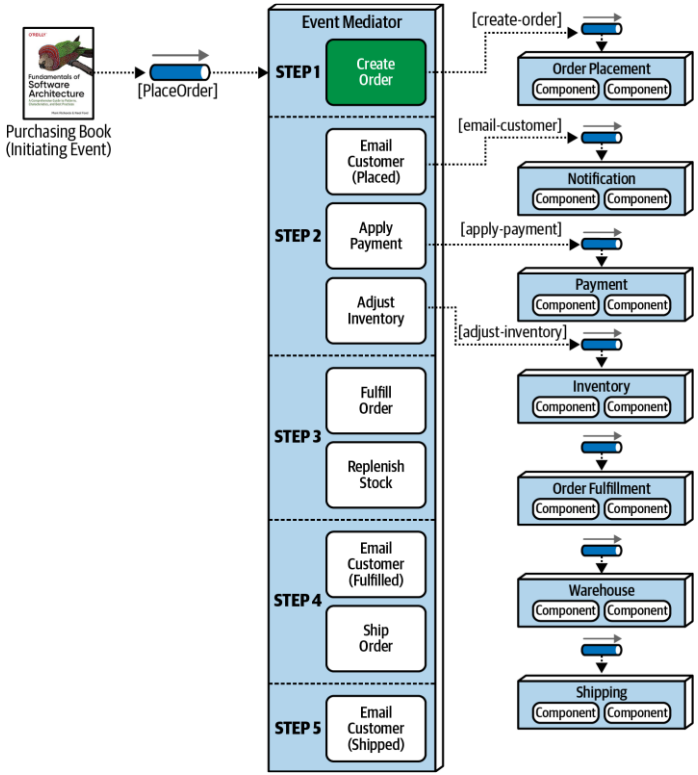
Mediator Topology – Example



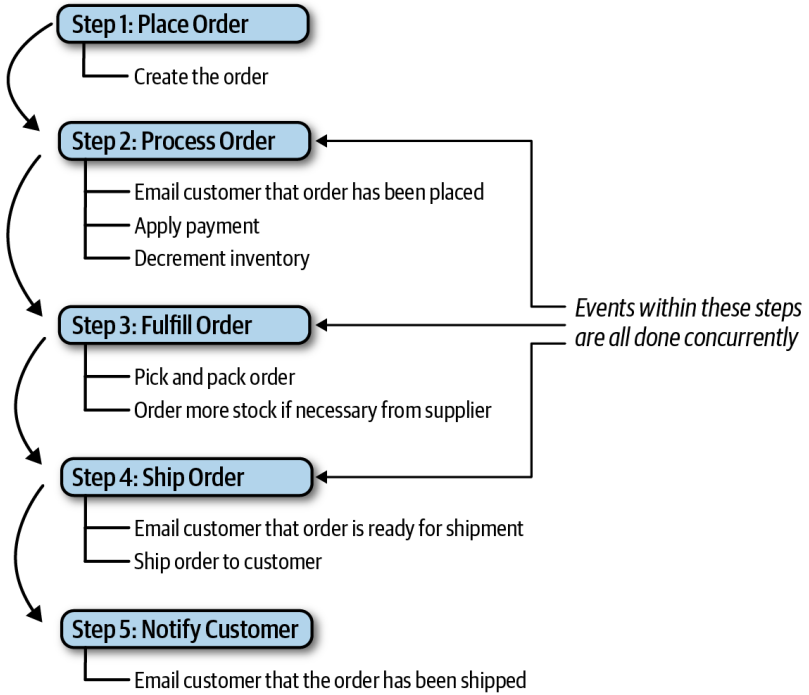


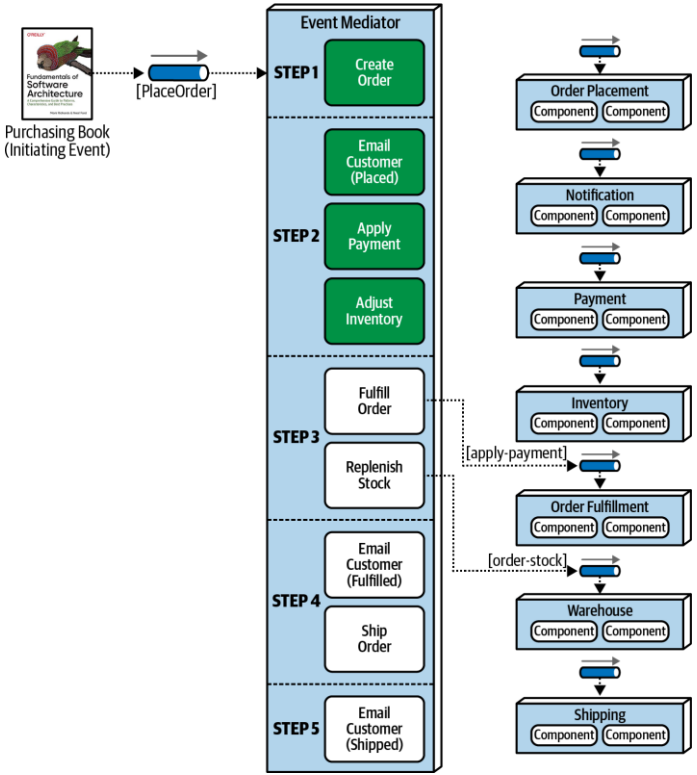
Mediator Topology – Example



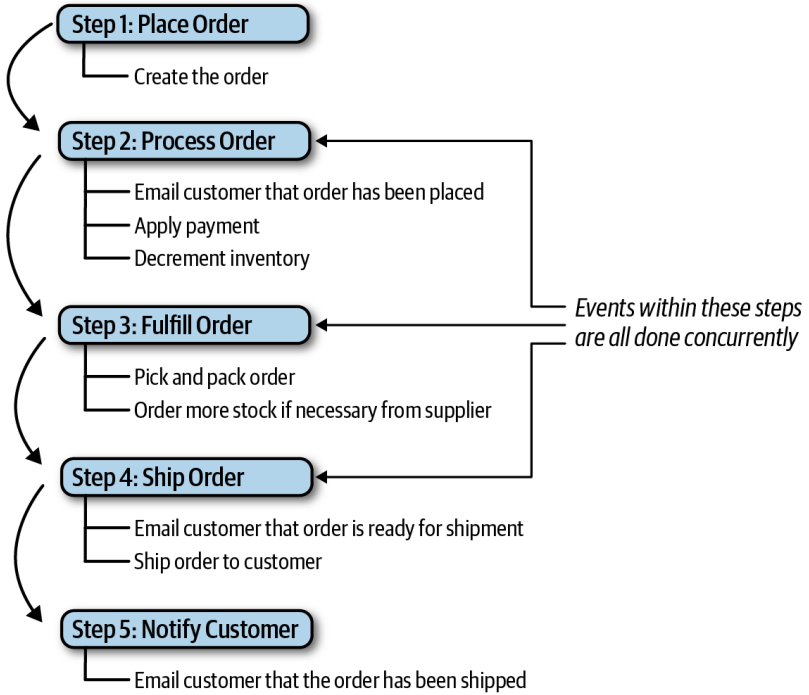


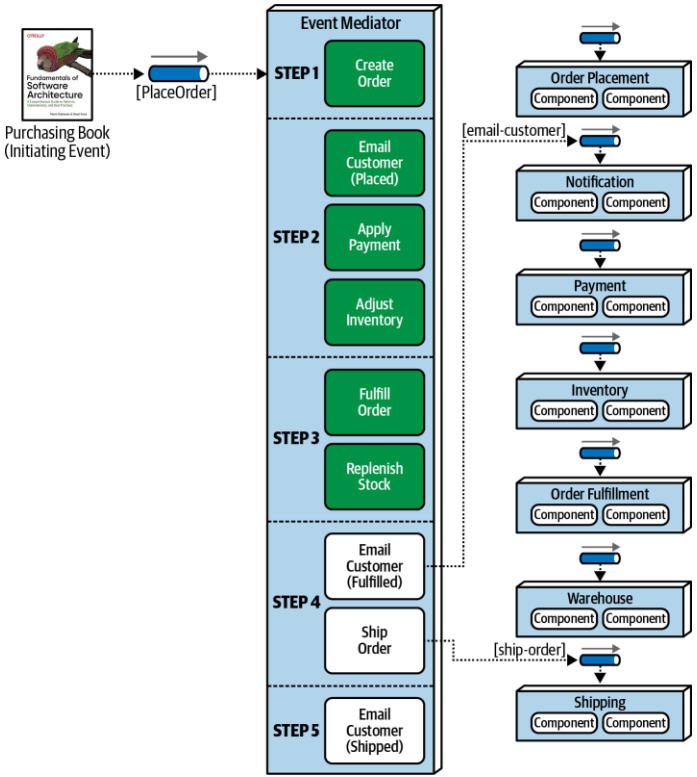
Mediator Topology – Example



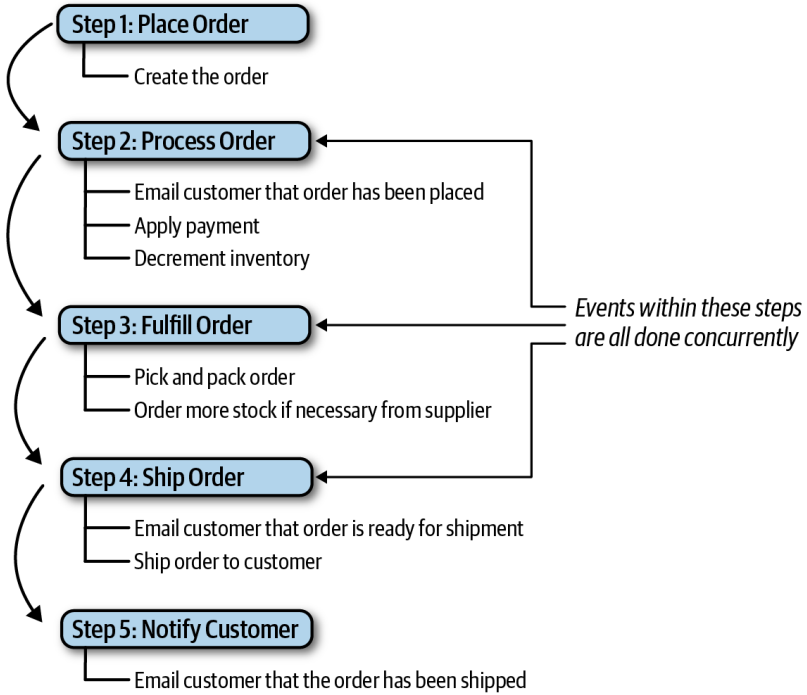


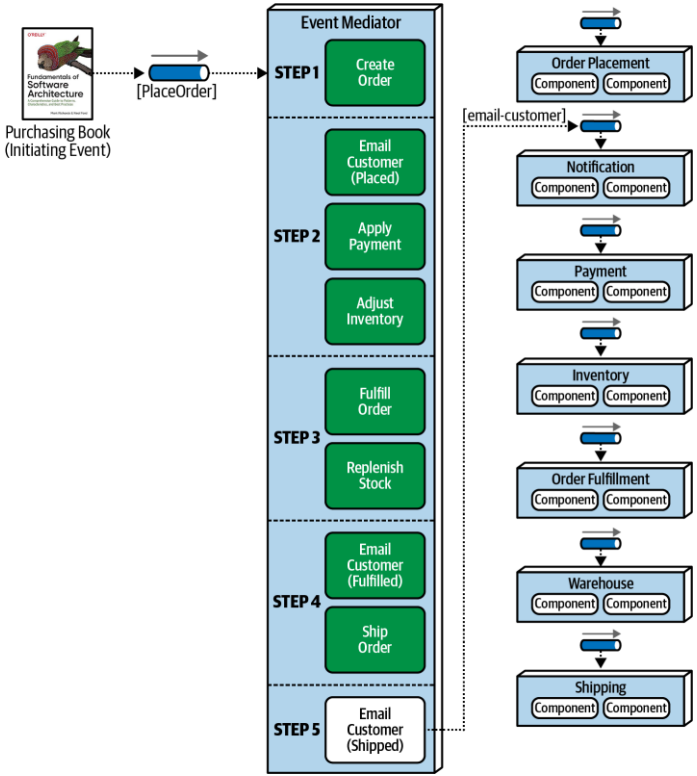
Mediator Topology – Example





Mediator Topology – Example







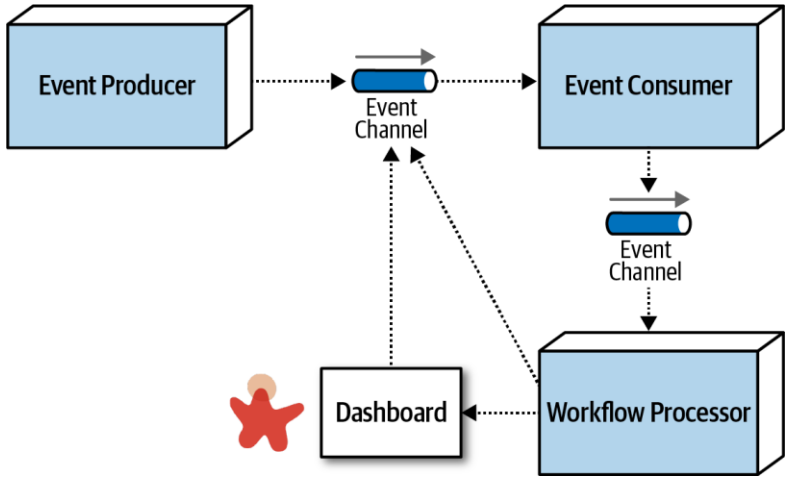
Mediator Topology – Trade-offs

- **Benefits:**
- The Mediator is responsible controlling the workflow and controlling it.
  - It can maintain the workflow state, allowing the processes to the paused/resumed.
  - It can perform error handling
  - Provides support for recoverability and restarting the process

- **Negatives:**
  - The mediator is typically a performance bottleneck. Performance can be good, but it is worse than with brokers.
  - Event processors are not as decoupled as in the broker topology.

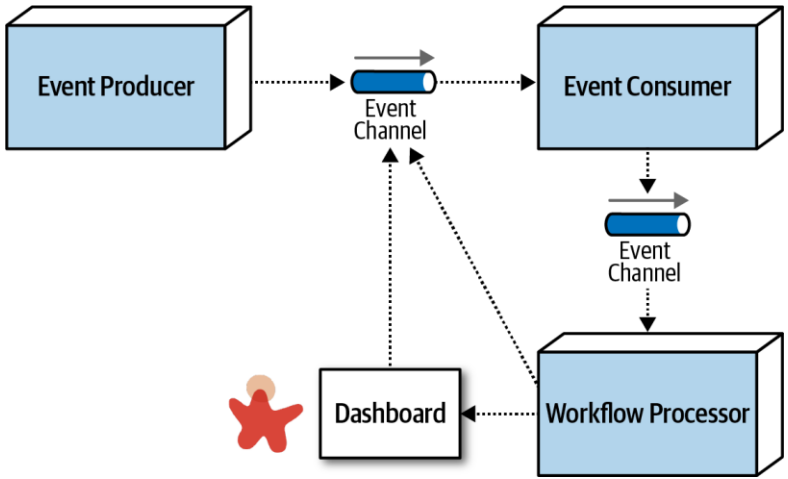
Error Handling – The Workflow Event Pattern

- The asynchonous operation in Event-Driven driven architectures helps to improve responsiveness, but complicates error handling.
- The **workflow event pattern** helps addressing this issue. It provides a standard way of dealing with errors in asynchronous contexts.
  - When some error is encountered, event consumers generate an event signalling an error instead of attempting to recover.
  - A Workflow processor will consume these events and is responsible for managing errors (storing, reporting, etc) and resubmitting events to the proper event channel.



Error Handling – The Workflow Event Pattern

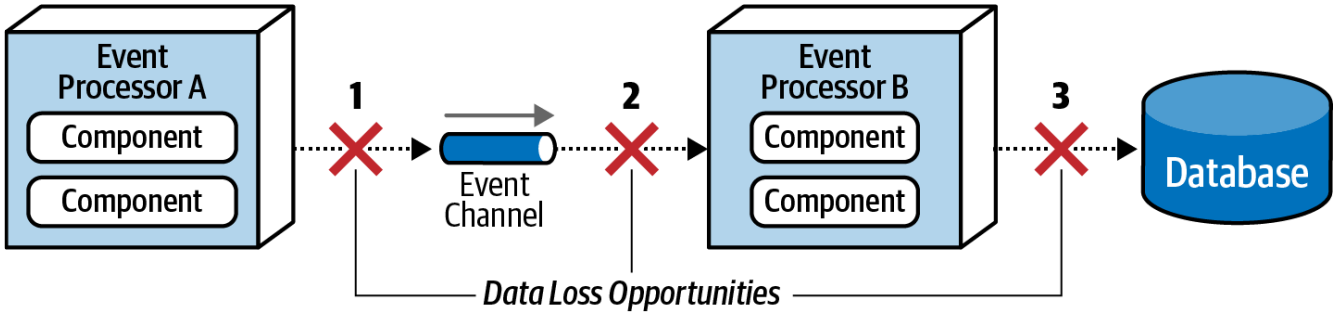
- When a workflow event is generated, the Workflow processor will try to figure out what is wrong and will attempt corrective measures. These can include:
  - Static analysis of request information using domain expertise.
  - Use of machine learning algorithms to detect anomalies in the data
- The workflow processor can make any necessary changes and resubmit the event.
- If no corrective measure is found, an error information is sent to a dashboard or similar component for reporting purposes.





Preventing Data Loss

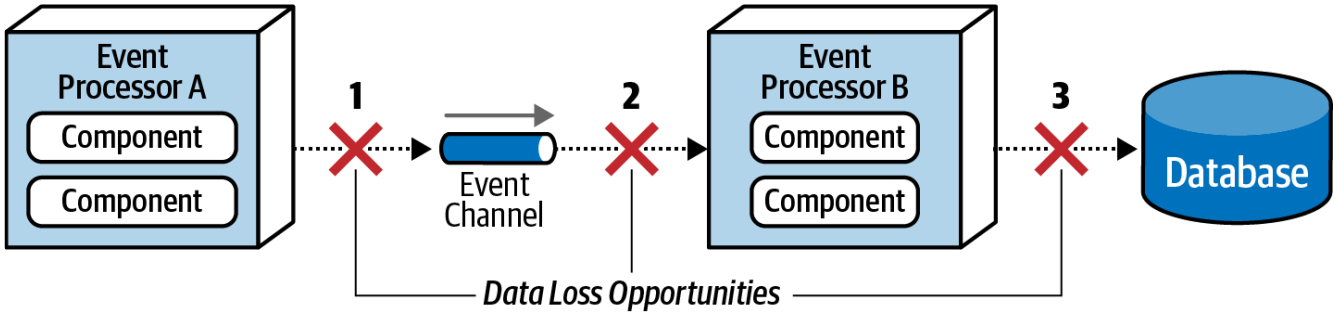
- There are ample opportunities for data loss in event-driven architectures.



1. Message does not make it to queue, or broker goes down before it is retrieved.
2. Event Processor B de-queues message, but crashes before processing it.
3. Attempt to persist data fails for some reason

Preventing Data Loss

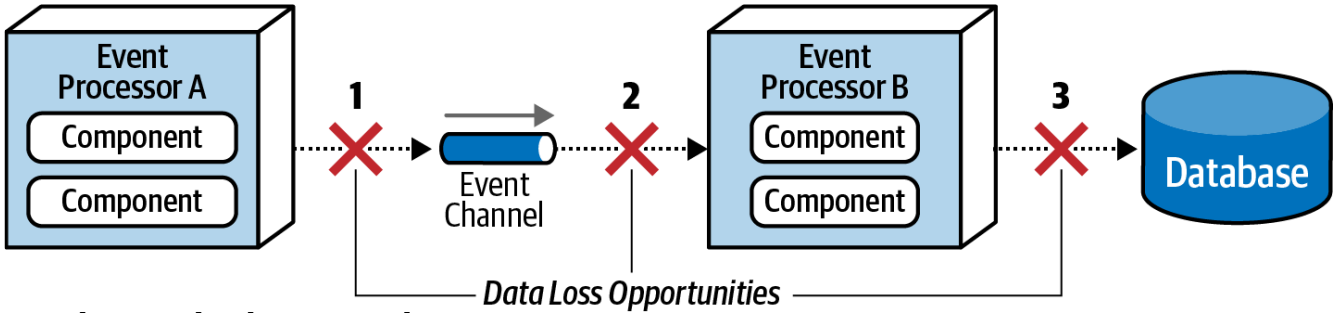
- Each of these issues can be mitigated by out-of-the-box techniques



1. **Persistent message queues with synchronous send.**
   - Broker stores incoming messages both in memory *and* physical data store, allowing recovery of undelivered messages if broker goes down.
   - Synchronous send blocks the message producer until the message is persisted.

Preventing Data Loss

- Each of these issues can be mitigated by out-of-the-box techniques

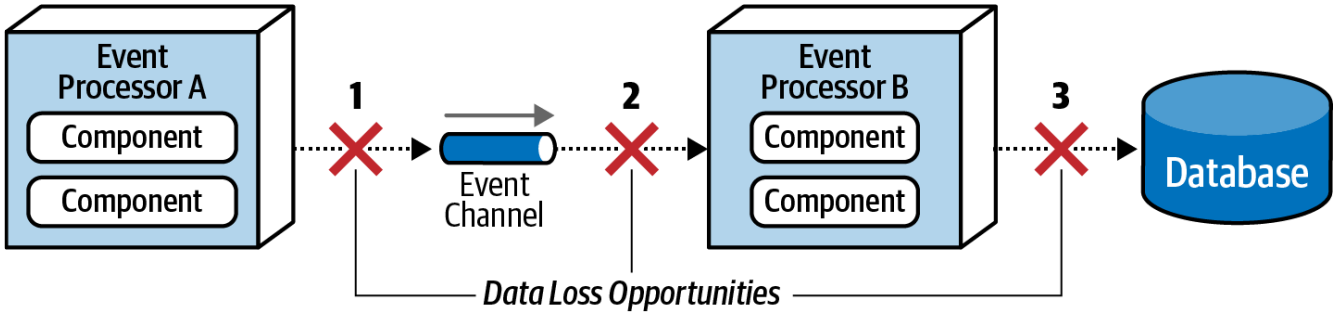


2. Client-Acknowledge Mode

- By default, de-queued messages are removed from the queue (*auto acknowledge mode*).
- In *Client-Acknowledge* mode, dequeued messages are kept on the queue and tagged with the client id, so they can be retransmitted later if necessary.

Preventing Data Loss

- Each of these issues can be mitigated by out-of-the-box techniques



2. **Last participant support**
   - Leverage ACID transactions in database to ensure successful persistency.
   - *Last participant support* – Acknowledge messages (removing them from persistent queue) only after successful transaction.

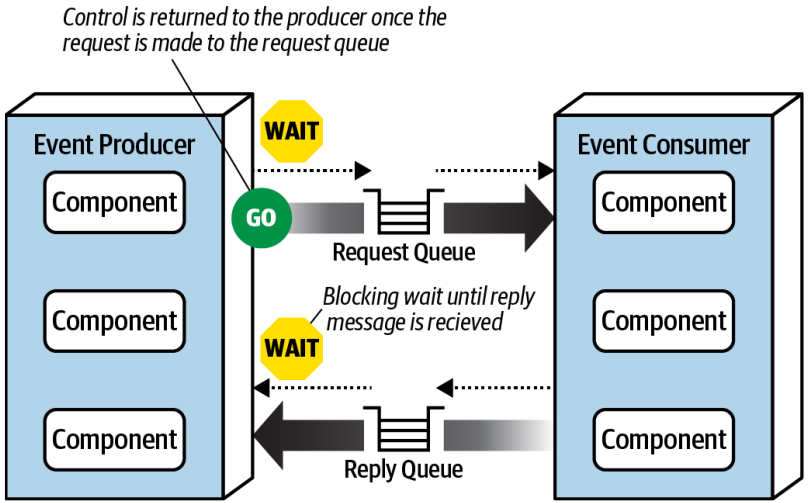


Request-Reply Messaging

- Event-driven architectures are mostly based on asynchronous communication channels.
- What about scenarios where a response is necessary (i.e. a synchronous behavior is desired):
  - *For example, we need to retrieve the generated order ID when creating a new order…*
- These scenarios can be dealt with by resorting to *request-reply* messaging.
  - Also known as pseudosynchronous communication

Request-Reply Messaging

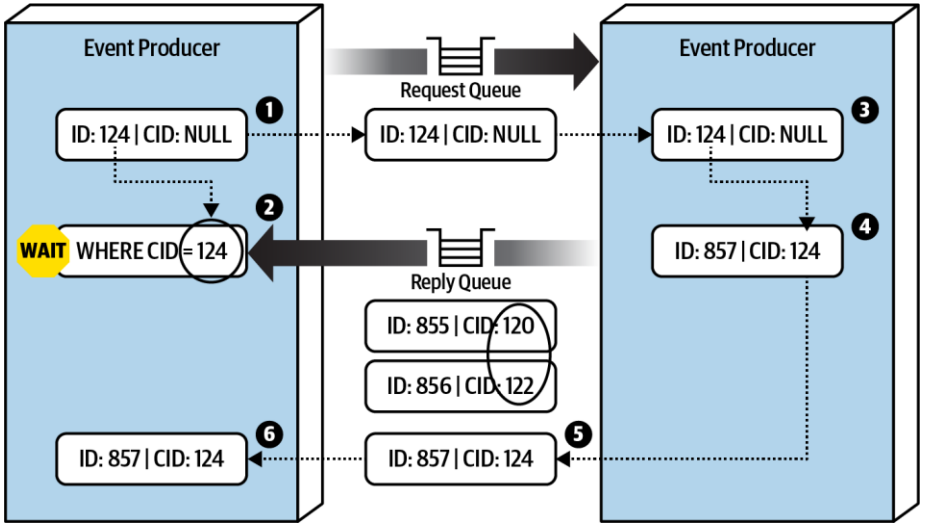
- **In request-reply messaging, each event channel is implemented with two queues:**
  - A request queue
  - A reply queue
- The producer sends the request asynchronously to the request queue.
- It then blocks waiting on the reply queue, until a reply message arrives.





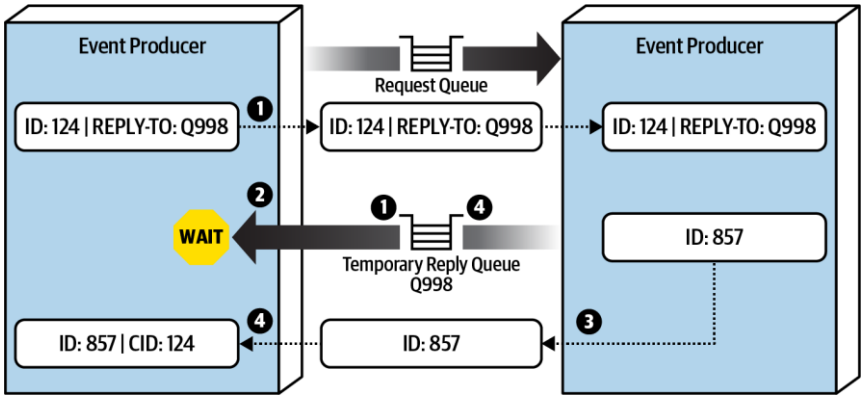
Request-Reply Messaging

- There are two possible approaches for managing the Request-Reply queues:
- The most common is the use of a *correlation-id* (CID).
  - A field in the response message indicates which original request it is concerned with.
  - The Producer does a blocking wait with a message filter/selector matching the expected CID.



Request-Reply Messaging

- **There are two possible approaches for managing the Request-Reply queues:**
- **Another approach is using temporary queues.**
  - For each request, a temporary queue is created specifically for returning the expected reply. It is deleted aftwards.
  - The producer creates the temporary queue and sends its name to the consumer.
  - A simple approach, but with large messaging volumes there can be negative impact on responsiveness and performance.





Request-based or Event-based

- **How to decide between a request-based (mediator) or event-based (broker) architecture:**
  - The following table highlights the trade-offs of the event-model

Advantages over request-based	Trade-offs
Better response to dynamic user content	Only supports eventual consistency
Better scalability and elasticity	Less control over processing flow
Better agility and change management	Less certainty over outcome of event flow
Better adaptability and extensibility	Difficult to test and debug
Better responsiveness and performance	
Better real-time decision making	
Better reaction to situational awareness	



Architectural Characteristics

- Technically partitioned - A domain is partitioned across multiple event processors tied by mediators, brokers, queues and topics.
- Number of quanta is aligned with the number of database interactions. (e.g. single-database, 1 quantum).
- Performance, scalability and fault tolerance are excellent.
- Simplicity and testability are low, due to non-deterministic and dynamic event flows.
  - Mediator-based architectures are better than broker-based models in this aspect.
- Highly evolvable, new events and event processors can be easily added.
  - Especially if using a broker topology.

Architecture characteristic	Star rating
Partitioning type	Technical
Number of quanta	1 to many
Deployability	★★★
Elasticity	★★★
Evolutionary	★★★★★
Fault tolerance	★★★★★
Modularity	★★★★
Overall cost	★★★
Performance	★★★★★
Reliability	★★★
Scalability	★★★★★
Simplicity	★
Testability	★★

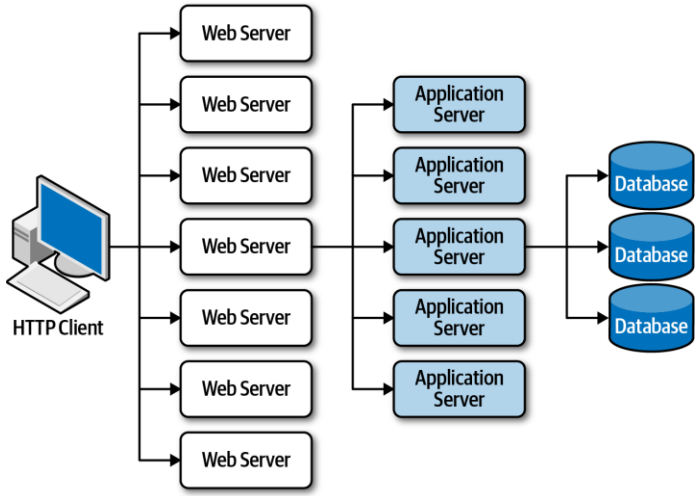
2.8

Space-Based Architecture Style

An architectural style focused on scalability, elasticity, and high levels of concurrency.

Space-Based Architecture Style

- **Consider a typical web-based business application involving a web server, application server, and database.**
  - As the load increases, bottlenecks appear, first at web server, then at the application server, then at the database.
- **Scaling out the system in this order is the natural response.**
  - Scaling the web server is fairly easy and inexpensive
  - But it might just move the bottle neck to the application server, which is not as easy nor inexpensive to scale.
  - And the same repeats for the database system, with increasing complexity and associated costs.
- **This leads to a "triangle-shaped" scaled topology.**





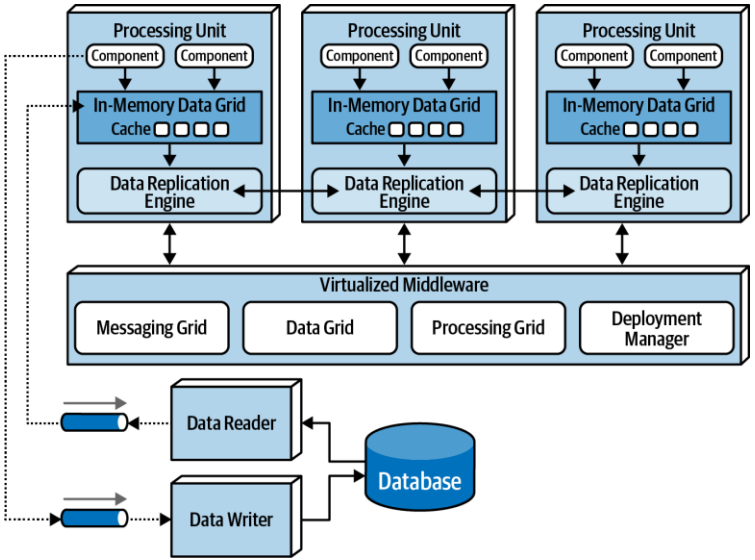
Space-Based Architecture Style: Overview

- **The Space-Based architectural style has its roots in systems with multiple parallel processors communicating with shared memory. Basic principles of operation of these systems include:**
  - No synchronization of access to a centralized data store, as this would strongly limit concurrency.
  - Application data is maintained in memory and replicated in all processing units.
  - Updated data is sent asynchronously to the data store.

- **By removing the need to synchronize data with a centralized database, the database bottleneck is removed, dramatically improving scalability.**

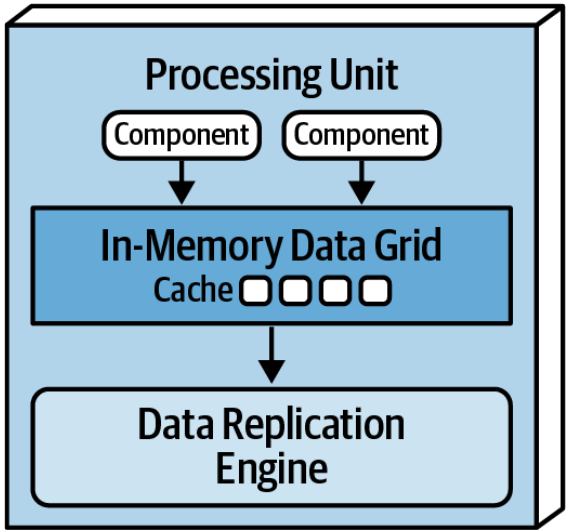
Space-Based Architecture Style: Components

- **The Space-Based architectural style includes the following components:**
  - **Processing Unit:** Contains application code.
  - **Virtualized Middleware:** Used to manage and coordinate the processing units.
  - **Data pumps:** Asynchronously send data to the database.
  - **Data writers:** Perform the updates from the data pumps.
  - **Data readers:** Read database data and deliver it to processing units upon startup.



Space-Based Architecture Style

- **Processing unit:**
  - Contains application logic (both web components AND backend).
  - PUs may be divided according to functional areas of the application.
  - A single PU might be sufficient for a simple application.
  - Contains an in-memory Data Grid, usually implemented with products such as Hazelcast, Apache Ignite or Oracle Coherence.



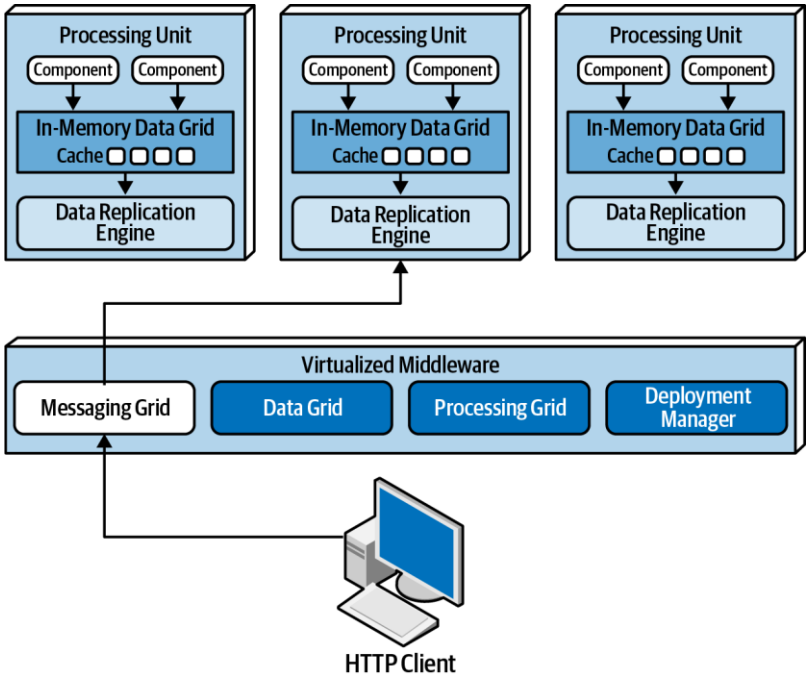


Space-Based Architecture Style

- **Virtualized Middleware:**
  - Handles infrastructure concerns.
  - Controls data synchronization and request handling.
  - Includes several components:
    - a messaging grid,
    - a data grid,
    - a processing grid,
    - a deployment manager.

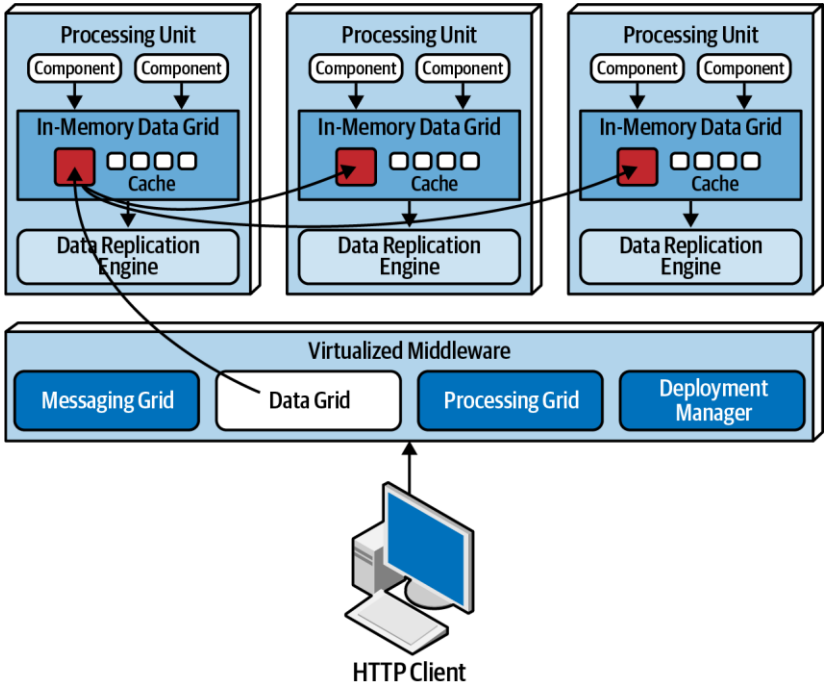
Virtualized Middleware

- **Messaging Grid:**
  - Manages input request and session state.
  - Receives requests and determines which specific PU it should be forwarded to.
  - Complexity might range from simple round-robin dispatch to more elaborated algorithms based on projected availability.
  - Usually implemented with a web-server with load balancing capabilities (HA Proxy or nginx).



Virtualized Middleware

- **Data Grid:**
  - A crucial component in this style of architecture.
  - Usually implemented as a replicated cache existing solely in PUs.
  - But if the implementation requires an external controller or if a distributed cache is used instead, it resides on this component in the middleware.
  - The in-memory data grid of all PUs should contain the same data. This data replication occurs asynchronously.





Virtualized Middleware

Data Grid:

- Data is synchronized between processing units that contain the same named data grid.
- An example with Hazelcast would be:

```
HazelcastInstance hz = Hazelcast.newHazelcastInstance();
Map<String, CustomerProfile> profileCache =
hz.getReplicatedMap("CustomerProfile");
```

- All processing units that manipulate the costumer profile would include this code, and all changes would be replicated across all other processing units also using the same named cache (“CustomerProfile”)
- Other alternatives include choreography or orchestration (will be analyzed later).



Virtualized Middleware

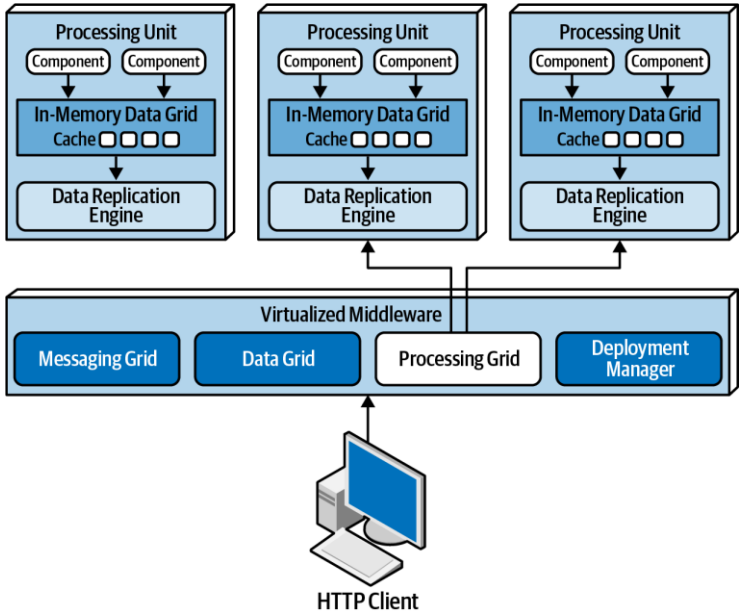
Data Grid:

- When a PU instance is created, its data is initialized by copy of another running instance having the required name cache, if it exists. This avoids accessing the physical database.
- A *member list* is used to make sure each PU instance is aware of other PU instances (IP and port). This list is updated and replicated across all instances using the same named cache.

```
Members {size:2, ver:4} [
Member [172.19.248.89]:5701 - 04a6f863-dfce-41e5-9d51-9f4e356ef268
Member [172.19.248.91]:5703 - 1623eadf-9cfb-4b83-9983-d80520cef753 this
]
```

Virtualized Middleware

- **Processing Grid:**
  - An optional component that orchestrates request processing across multiple Pus if necessary.
  - For example, if a request requires coordination between an order processing unit and a payment processing unit, the processing grid is responsible for mediating and orchestrating that response.



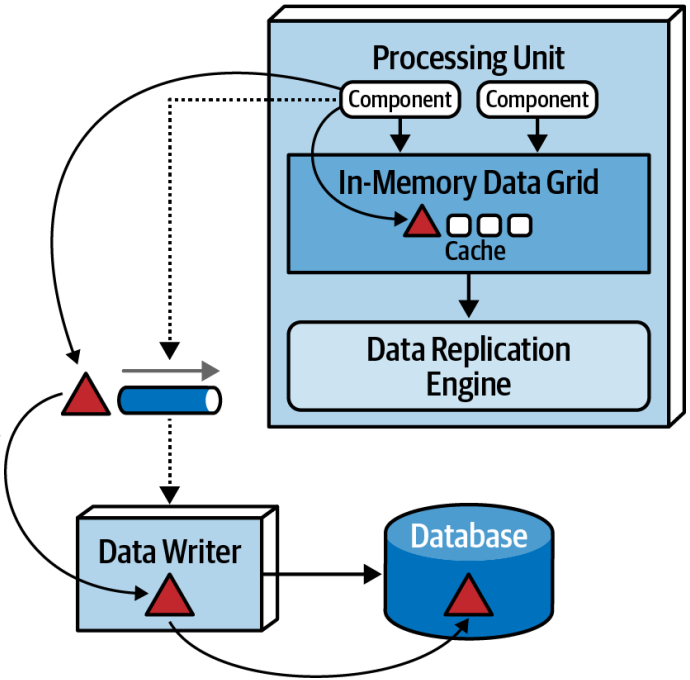


Virtualized Middleware

- **Deployment Manager:**
  - Manages dynamic startup and shutdown of Processing Units based on load conditions.
  - Monitors response times and loads and starts or shuts down processing unites as necessary.
  - Critical for achieving elasticity.

Data Pumps & Data Writers

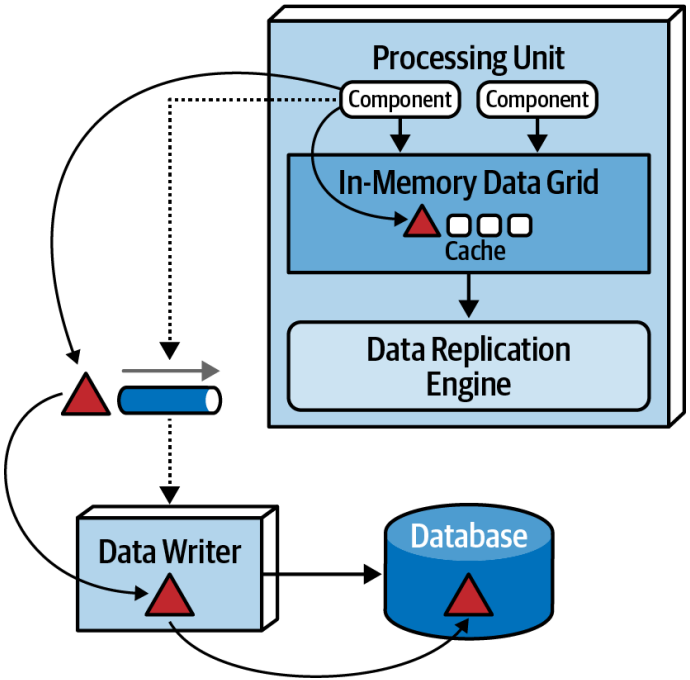
- **A Data Pump sends data to be updated in the database.**
  - Operates asynchronously.
  - It ensures eventual consistency of in-memory cache and physical database.
  - Processing units are responsible for sending updated data via the data pump, so that it is eventually persisted in the physical database.
  - Usually implemented with messaging.
  - There are usually multiple data pumps, associated with particular domains (e.g. costumer or inventory) or named caches.





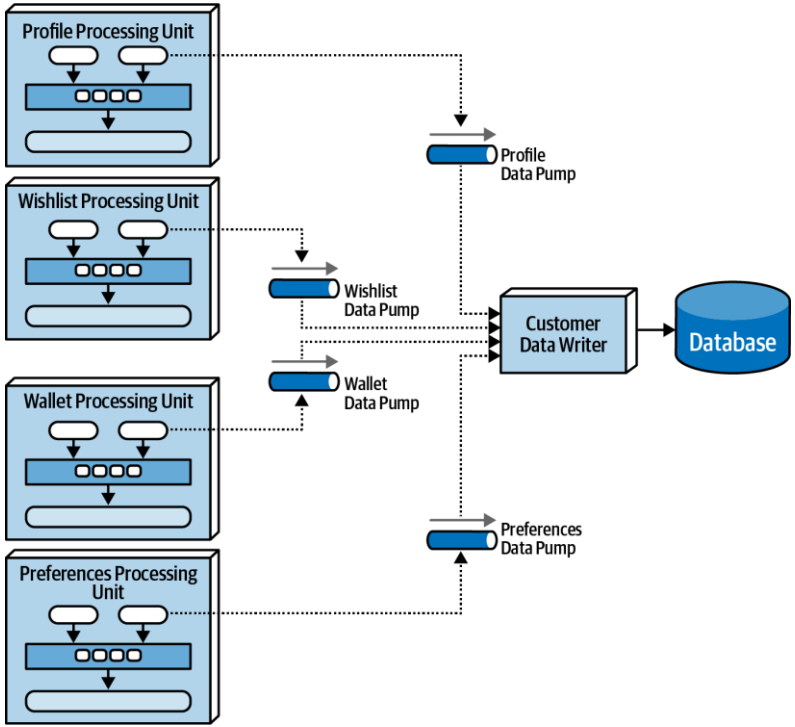
Data Pumps & Data Writers

- **A Data Writer accepts messages from a data pump and stores them in a physical database.**
- The granularity of the data writers can vary based on the scope of the data pumps and processing units.



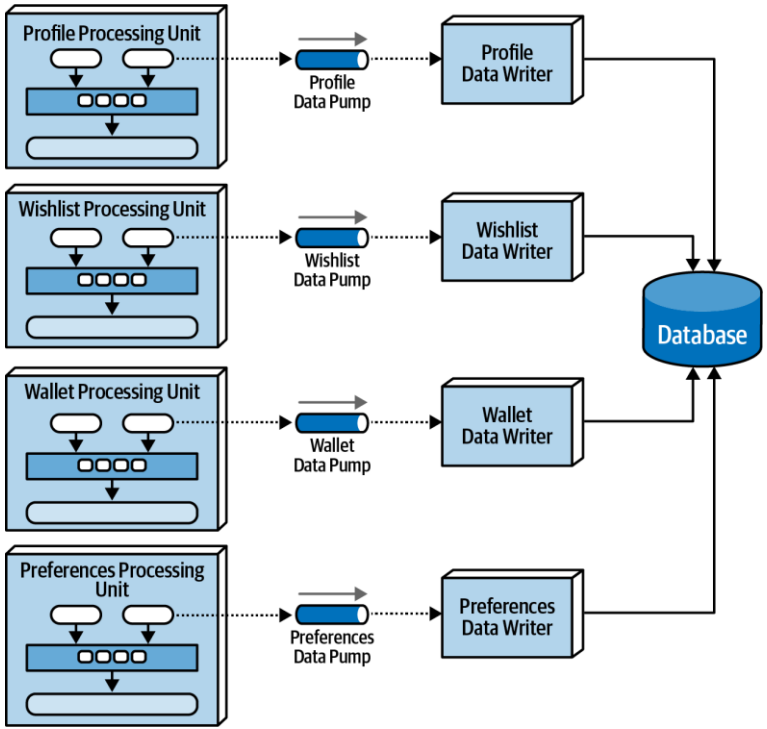
Data Writer – Domain Based

- A domain-based data writer contains all of the necessary database logic to handle all the updates within a particular domain (regardless of the number of data pumps).



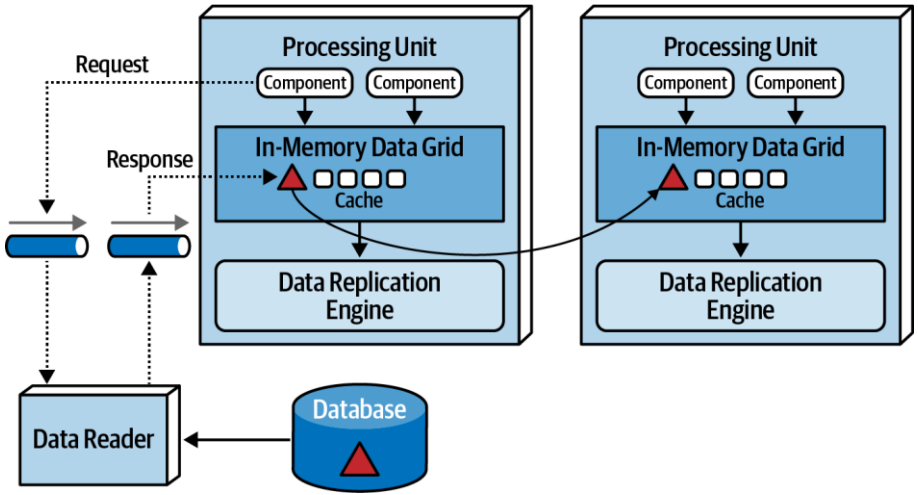
Data Writer – Dedicated

- Alternatively, each class of processing unit might have a dedicated data writer
- Requires more Data Writer components
- Allows better scalability and agility



Data Readers

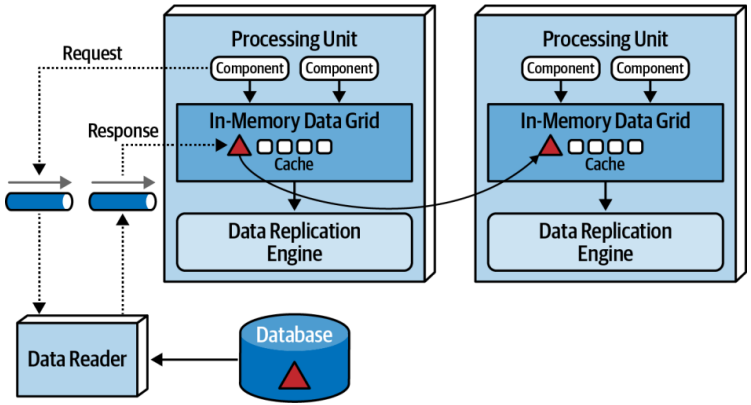
- A Data Reader has the responsibility of reading data from the database and sending it to the Processing Units with a reverse data pump.
- Only invoked in three scenarios:
  - A crash of all processing unit instances of the same named cache.
  - A redeployment of all processing units within the same named cache.
  - Retrieving data not contained in the replicated cache.





Data Readers

- **When all instances came down (system wide crash or redeployment) and are restarted:**
- All PUs try to acquire a cache lock.
- The first one to get it becomes the temporary cache owner.
- It then sends the Data Reader a message requesting the data.
- The Data Reader will respond, and when all data is read, the lock is released and all other PUs waiting on the lock are updated from the in-memory data.





Data Colisions

- Latency in the replication of data across in memory caches may causa **data colisions.**
- A data collision occurs when data is updated in one cache instance (Cache A), and later, during replication, the same data is updated in another cache instance (Cache B).
  - The local update to Cache B will be overridden by the replication of old data from Cache A.



Data Colisions

- Example Scenario:
  1. Current shared cache Stock is 500
  2. PU instance A – Updates local Stock to 495, while processing an order (purchase of 5 units). Asynchronous replication of the update is triggered.
  3. PU instance B – Updates local Stock to 490, while processing an order (purchase of 10 units). Asynchronous replication of the update is triggered.
  4. PU instance B – Local Stock is updated to 495, upon conclusion of replication from PU instance A.
  5. PU instance A – Local Stock is updated to 490, upon conclusion of replication from PU instance B.
  6. Both PUs end up with incorrect stock (should be 485).



Data Colisions

- Data collisions are unavoidable in replicated caches
- Potential data collisions can be estimated probabilistically by the formula:

$$\text{CollisionRate} = N^* (UR^2/S) * RL$$

Where:

- N – Number of instances using the same named cache,
- UR – Update rate in milliseconds,
- S – Cache size (number or rows),
- RL – Replication latency of the cache.



Data Colisions

- Data collisions are unavoidable in replicated caches
- Potential data collisions can be estimated probabilistically by the formula:

$$CollisionRate=N^* (UR/S) * RL$$

Where:

- Cr – Collision Rate (% of data collisions)
- N – Number of instances using the same named cache,
- UR – Update rate in milliseconds,
- S – Cache size (number or rows),
- RL – Replication latency of the cache.



Data Colisions

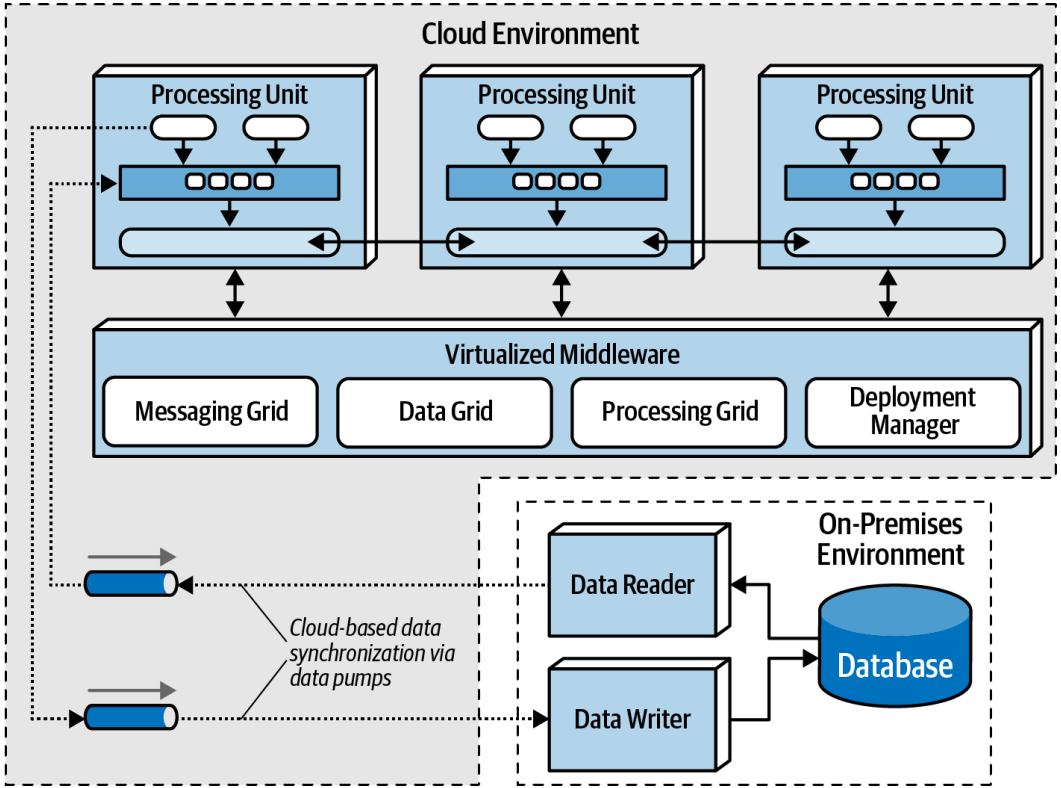
- Example:
  - Update Rate (UR) – 20 updates/second
  - Number of Instances (N) – 5
  - Cache Size (S) – 50000 rows
  - Replication Latency (RL) – 0.1 seconds (100 milliseconds)

Updates: 72000 updates / hour (3600*UR)

Collision Rate: 14.4 collisions / hours

Deployment

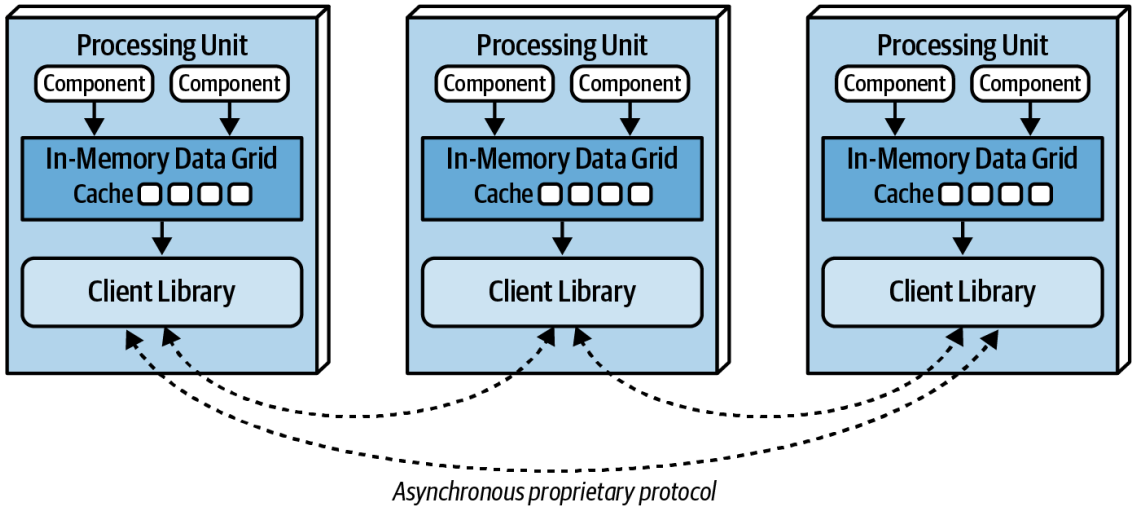
- Deployment is very flexible, as all architectural components can deployed in the cloud or on-premises (locally/non-cloud).
- A hybrid approach can also be used and is common.





Replicated Caching vs Distributed Caching

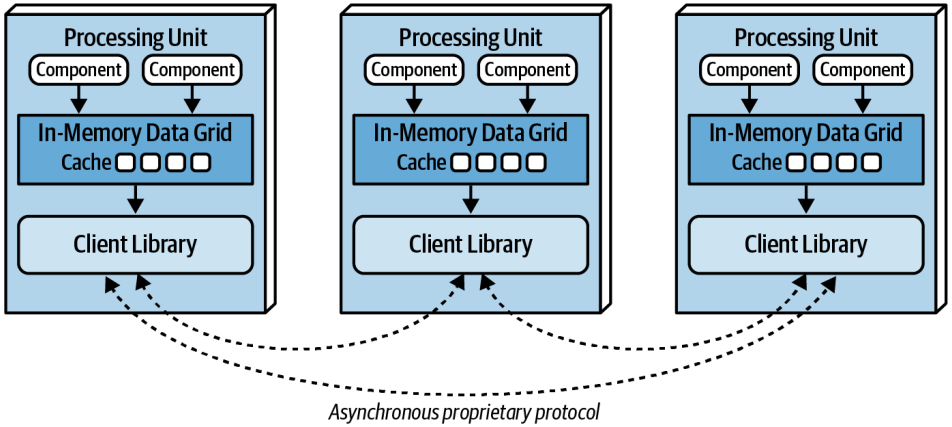
- We considered a *replicated cache* in our example so far:
  - All processing units have their own local copy of the data, with changes being propagated asynchronously.





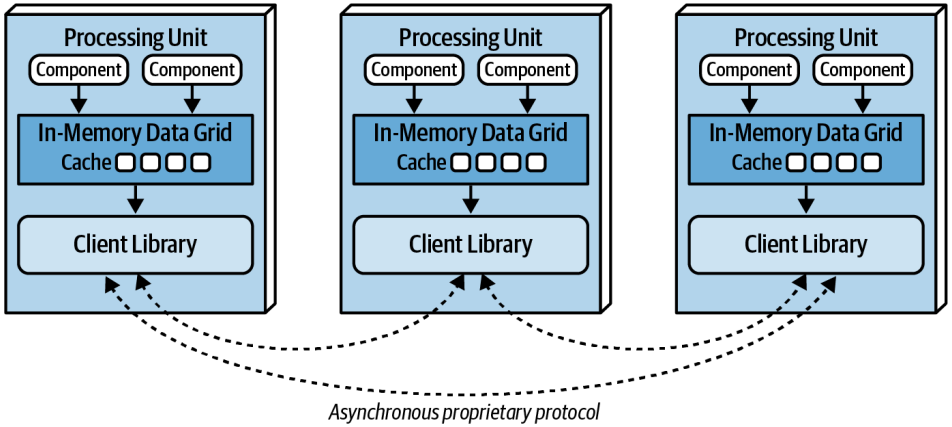
Replicated Caching vs Distributed Caching

- We considered a *replicated cache* in our example so far:
  - All processing units have their own local copy of the data, with changes being propagated asynchronously.
  - Very fast due to in-memory access.
  - Usually has a high level of fault-tolerance (in most implementations, there is no single point of failure)



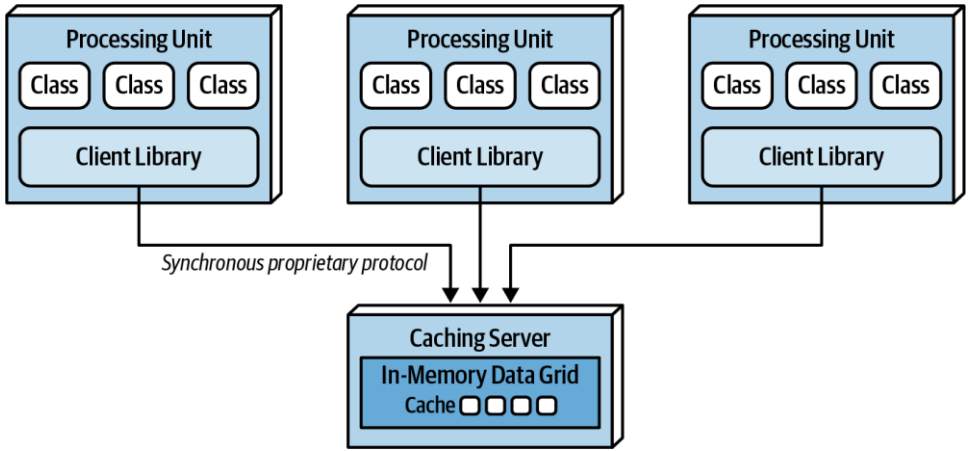
Replicated Caching vs Distributed Caching

- **Replicated Caching has some disadvantages:**
  - Does not handle well high data volumes (cache sizes) with high update rates.
  - Large in-memory caches (over 100 Mb) may impair scalability and elasticity due to high memory requirements in each PU.
  - Consistency is not ensured due to data collisions. This problem is exacerbated for high update rates.



Replicated Caching vs Distributed Caching

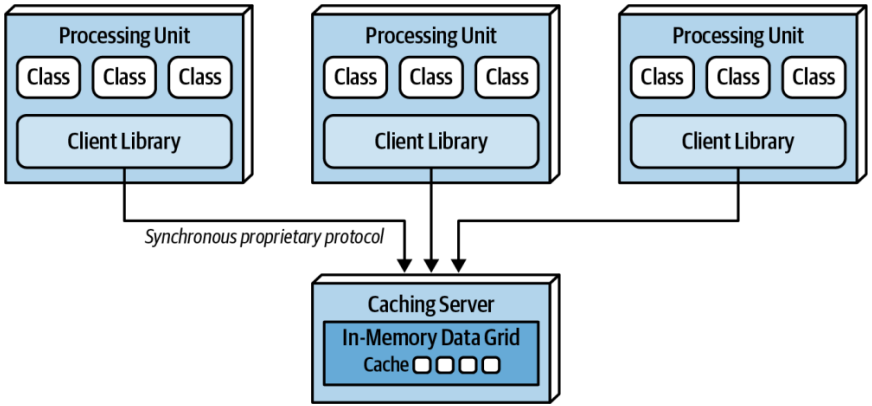
- **Distributed Caching is an alternative that addresses these shortcomings.**
  - An external server or service is dedicated to holding a centralized cache.
  - PUs do not have a local data grid, and must connect to the central cache server to access the data.
  - Data consistency is very high because no replication is required.





Replicated Caching vs Distributed Caching

- **Of course, Distributed Caching has its own limitations.**
  - Performance is worse, due to external access, adding latency to the system.
  - Fault tolerance is inherently lower because the caching server is a single point of failure. If it goes down, the entire system becomes non-operational.
    - Replication of the caching server helps, but introduces consistency issues. No free lunch…





Replicated Caching vs Distributed Caching

- For caches of very high dimension (>500 Mb) or with very high update rates, a distributed cache may be a better option.
- For relatively small caches (<100 Mb) and low update rates, the choice between replicated or distributed caches corresponds to a choice between better data consistency (with a distributed cache) or better performance and fault tolerance (with a replicated cache)

Decision criteria	Replicated cache	Distributed cache
Optimization	Performance	Consistency
Cache size	Small (<100 MB)	Large (>500 MB)
Type of data	Relatively static	Highly dynamic
Update frequency	Relatively low	High update rate
Fault tolerance	High	Low



Replicated Caching vs Distributed Caching

- **Both models may be applicable in the same application, because different sets of data may have different properties, and we can leverage the benefits of both approaches.**
  - Of course, simplicity and cost suffer…
- **For example, we can have different systems for different processing units in the same application:**
  - A processing unit that maintains Costumer Profile Information does not have high consistency requirements and is mostly static with a very slow update rate. A replicated cache will be a good option here for performance and fault tolerance.
  - A processing unit that maintains the current inventory data requires high consistency. Therefore, a distributed cache may be a better option in this case.



Architectural Styles

- To be continued next class....